

Deep Learning Laboratory

Assignment #5

Siddharth Narayan Mishra
123CS0189

September 2, 2026

Contents

How to read this report	3
Experiment 1: Greedy Layer-wise Pre-training	3
1 Objective	3
2 Theory	3
2.1 The problem this method was invented to solve	3
2.2 Autoencoders in one paragraph	4
2.3 The greedy recipe used in Case 2	4
2.4 The supervised loss	4
3 Algorithm	5
4 Implementation	5
5 Output	7
6 Observation	8
7 Conclusion	8
Experiment 2: Regularization on Letter Recognition	10
8 Objective	10
9 Theory	10
9.1 Generalisation, overfitting, underfitting	10
9.2 Softmax and sparse categorical cross-entropy	10
9.3 The seven cases, each in a sentence or two	11
9.4 Why report the weight norm	11

10 Algorithm	12
11 Implementation	12
12 Output	13
13 Observation	15
14 Conclusion	16

How to read this report

This report is written as a study guide rather than a bare lab writeup. Each experiment starts with the plain idea, then the mathematics needed to follow the code, then the code itself, then the numbers we actually got, and finally an honest reading of what those numbers mean. Where a result looks disappointing, the report explains why, because understanding why a method fails to help is just as useful as seeing it succeed.

Both experiments were implemented from scratch using only `numpy`, `pandas` and `matplotlib`. There is no deep learning framework involved: every forward pass, gradient, weight update, dropout mask and softmax is written out by hand in `exp1.py` and `exp2.py`.

Settings shared by every model. Sigmoid activation in the hidden layers. Inputs standardized with a `StandardScaler` that is fitted on the training split only (so the model never peeks at test statistics). Weights initialized uniformly at random in $[-0.5, 0.5]$, biases initialized to zero. Optimizer is stochastic gradient descent with momentum, learning rate 0.001, momentum 0.8. Random seed 42. Thirty supervised epochs.

Experiment 1: Greedy Layer-wise Unsupervised Pre-training

1 Objective

- Build a deep classifier $X \rightarrow H_1 \rightarrow H_2 \rightarrow H_3 \rightarrow H_4 \rightarrow Y$ with layer sizes $3 \rightarrow 3 \rightarrow 3 \rightarrow 3 \rightarrow 1$ for the Social Network Ads dataset (predict whether a user purchased, from Gender, Age and EstimatedSalary).
- Case 1: train this network directly with supervised learning and use it as the baseline.
- Case 2: first pre-train each hidden layer one at a time, without using the labels, as an autoencoder; then stack the pre-trained layers, attach the output neuron, and fine-tune the whole thing with the labels.
- Compare the two training curves and understand when this two-stage recipe is expected to help.

2 Theory

2.1 The problem this method was invented to solve

Around 2006, training deep networks made only of sigmoid layers was hard. The reason is the *vanishing gradient*. The derivative of the sigmoid $\sigma(z) = 1/(1+e^{-z})$ is $\sigma(z)(1-\sigma(z))$, which is at most 0.25 and is much smaller than that whenever a neuron is even mildly saturated. Backpropagation multiplies one such factor per layer. Through four hidden layers the gradient reaching the first layer is scaled by roughly $0.25^4 \approx 0.004$ in the best

case and far less in practice. With a small learning rate the early layers therefore barely move, and the network never learns good low-level features.

Greedy layer-wise pre-training sidesteps this. Instead of trying to push a gradient signal all the way down from the output, we teach each layer to do something useful *locally*, one layer at a time, and only then train the full stack.

2.2 Autoencoders in one paragraph

An autoencoder is a network that is trained to copy its input to its output through a narrow middle. It has an encoder that maps the input to a hidden representation h , and a decoder that maps h back to a reconstruction $\hat{x}$. It is trained to minimise the reconstruction error, here the mean squared error

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N d} \sum_{i=1}^N \sum_{j=1}^d (\hat{x}_{ij} - x_{ij})^2.$$

Because h is forced to pass through a bottleneck, the encoder cannot just copy the numbers across. It has to find a compact code that keeps whatever structure is needed to rebuild the input. That code is exactly the kind of feature we would like a hidden layer to compute. No labels are used, so this is unsupervised learning.

2.3 The greedy recipe used in Case 2

1. **Stage 1.** Build an autoencoder $X \rightarrow H_1 \rightarrow \hat{X}$. Train it for 20 epochs on the training inputs. Keep the encoder weights W_1 (the part that produces H_1); throw the decoder away.
2. **Stage 2.** Freeze W_1 . Push every training example through it to get the representation H_1 . Build a new autoencoder $H_1 \rightarrow H_2 \rightarrow \hat{H}_1$ and train it for 20 epochs so that layer H_2 learns to reconstruct the output of layer H_1 . Keep W_2 .
3. **Stage 3 and Stage 4.** Repeat: $H_2 \rightarrow H_3$, then $H_3 \rightarrow H_4$. Every earlier layer stays frozen while the current one is trained.
4. **Assemble and fine-tune.** Stack W_1, W_2, W_3, W_4 into the deep network, add a fresh output neuron $H_4 \rightarrow Y$ with random weights, unfreeze everything, and train the whole network with the labels using binary cross-entropy for 30 epochs.

The intuition is that fine-tuning now starts from a point where every layer already computes something structured, instead of starting from random noise. It is a smarter initialization, not a different model.

2.4 The supervised loss

The output neuron uses a sigmoid, so it produces a probability $p \in (0, 1)$ that the user purchased. Training minimises binary cross-entropy

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right].$$

This is small when the predicted probability is close to the true label and grows without bound when the model is confidently wrong. Accuracy is reported by thresholding p at 0.5.

3 Algorithm

1. Load the CSV. Encode Gender as Female = 0, Male = 1. Inputs are [Gender, Age, EstimatedSalary], target is Purchased.
2. Make an 80:20 stratified train and test split (stratified means the purchased-to-not-purchased ratio is the same in both parts).
3. Fit `StandardScaler` on the training inputs, transform both splits.
4. **Case 1:** build the $3 \rightarrow 3 \rightarrow 3 \rightarrow 3 \rightarrow 3 \rightarrow 1$ network, train 30 epochs with BCE, record train and test loss and accuracy each epoch.
5. **Case 2:** run the four pre-training stages above, assemble the network from the learned encoders, then fine-tune 30 epochs exactly as in Case 1.
6. Plot loss and accuracy curves for each case and a direct comparison.

4 Implementation

A dense layer stores its weights, its bias, and one momentum buffer per parameter. Forward keeps the pre-activation z and the activation a so backward can use them.

```

1 class Dense:
2     def __init__(self, n_in, n_out, activation, rng):
3         self.W = rng.uniform(-0.5, 0.5, size=(n_in, n_out))
4         self.b = np.zeros(n_out)
5         self.activation = activation
6         self.vW = np.zeros_like(self.W)
7         self.vb = np.zeros_like(self.b)
8
9     def forward(self, x):
10        self.x = x
11        self.z = x @ self.W + self.b
12        self.a = sigmoid(self.z) if self.activation == "sigmoid"
13        else self.z
14        return self.a
15
16    def backward(self, da):
17        dz = da * self.a * (1.0 - self.a) if self.activation == "
18        sigmoid" else da
19        self.dW = self.x.T @ dz
20        self.db = dz.sum(axis=0)
21        return dz @ self.W.T
22
23    def step(self):
24        # SGD with momentum
25        self.vW = MOMENTUM * self.vW - LR * self.dW

```

```

23         self.vb = MOMENTUM * self.vb - LR * self.db
24         self.W += self.vW
25         self.b += self.vb

```

One pre-training stage builds a small autoencoder (sigmoid encoder, linear decoder so it can reconstruct real-valued standardized data), trains it, and returns only the encoder:

```

1 def pretrain_layer(representation, rng, epochs):
2     dim = representation.shape[1]
3     encoder = Dense(dim, 3, "sigmoid", rng)
4     decoder = Dense(3, dim, "linear", rng)
5     layers = [encoder, decoder]
6     for _ in range(epochs):
7         order = rng.permutation(len(representation))
8         for start in range(0, len(representation), BATCH_SIZE):
9             xb = representation[order[start:start + BATCH_SIZE]]
10            out = net_forward(layers, xb)
11            net_backward(layers, mse_grad(out, xb))    # target is
12                                                         the input
13            net_step(layers)
14    return encoder, losses

```

The Case 2 driver chains the stages, freezing earlier layers implicitly by feeding the next stage the already-computed representation:

```

1 representation = X_train.copy()
2 encoders = []
3 for stage in range(4):
4     encoder, _ = pretrain_layer(representation, rng,
5                                PRETRAIN_EPOCHS)
6     encoders.append(encoder)
7     representation = encoder.forward(representation)    # move to
8                                                         next layer
9
10 layers = [copy_as_dense(enc) for enc in encoders]    # reuse
11                                                         learned weights
12 layers.append(Dense(3, 1, "sigmoid", rng))    # fresh
13                                                         output neuron
14 history = train_supervised(layers, X_train, y_train, X_test,
15                             y_test, 30, rng)

```

5 Output

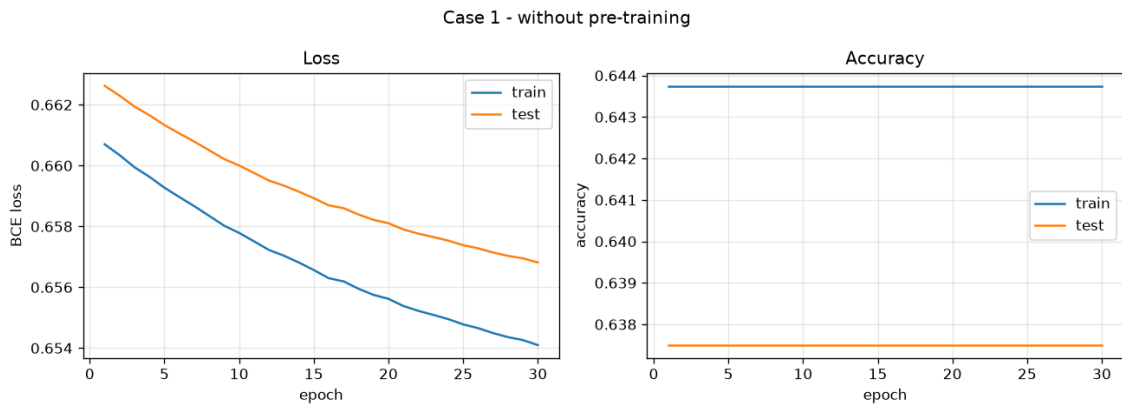


Figure 1: Case 1, no pre-training. Loss falls slowly; accuracy is a flat line.

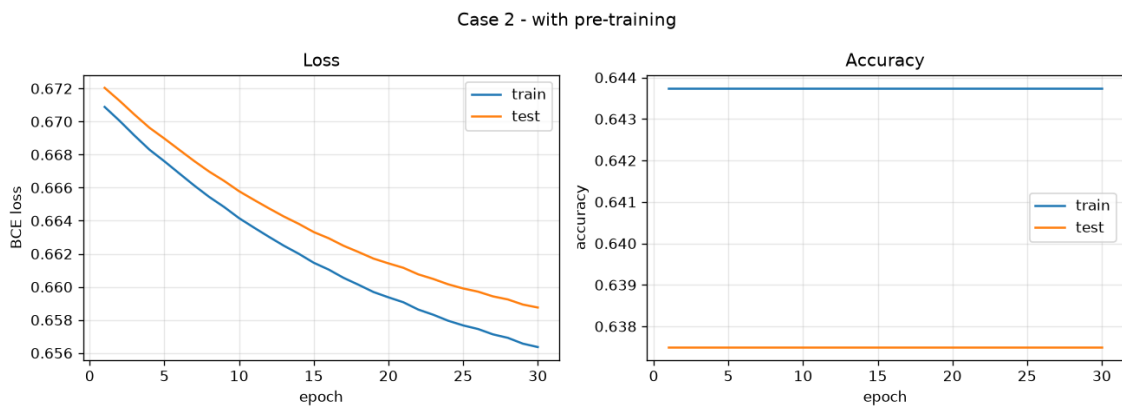


Figure 2: Case 2, with pre-training. The picture is essentially the same as Case 1.

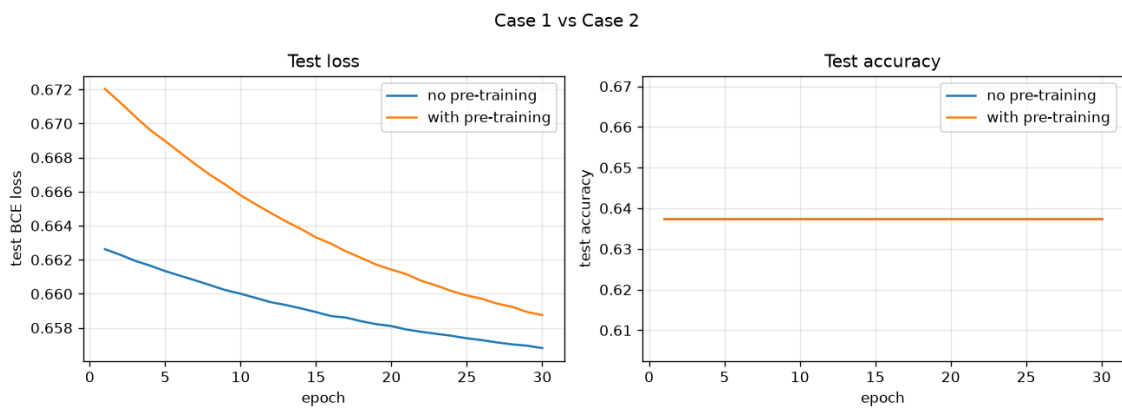


Figure 3: The two cases overlaid. The test-accuracy curves sit exactly on top of each other.

	train loss	test loss	train acc	test acc
Case 1, no pre-training	0.6541	0.6568	0.6438	0.6375
Case 2, with pre-training	0.6564	0.6588	0.6438	0.6375

Table 1: Final values at epoch 30.

The reconstruction errors during pre-training were:

stage	MSE at epoch 1	MSE at epoch 20
1: $X \rightarrow H_1$	1.0077	0.9729
2: $H_1 \rightarrow H_2$	0.2385	0.0451
3: $H_2 \rightarrow H_3$	0.1124	0.0100
4: $H_3 \rightarrow H_4$	0.1717	0.0134

Table 2: Greedy pre-training, per-stage reconstruction MSE.

6 Observation

- **Both models end up predicting the same single answer for everyone.** In the training set, 64.4% of users did not purchase. A model that always says “not purchased” scores 63.8% on the test set. Both Case 1 and Case 2 land on exactly that number and their accuracy curves are perfectly flat. The loss does drift down over the 30 epochs, which means the output neuron is nudging its bias to match the class ratio, but no model ever learns to separate buyers from non-buyers.
- **Why nothing learns.** Four narrow sigmoid layers, a learning rate of 0.001, and only 30 epochs together give the gradient no chance to reach the input side of the network. This is the vanishing-gradient situation described in the theory, reproduced on purpose by the fixed settings.
- **Pre-training did work at its own job.** Stages 2, 3 and 4 drove the reconstruction error down by one to two orders of magnitude, so those layers genuinely learned to represent their inputs. Stage 1 barely moved, because squeezing standardized real-valued data through a single sigmoid of the same width and rebuilding it with a linear decoder, at this tiny learning rate, is close to hopeless in 20 epochs.
- **Why pre-training still did not help the final accuracy.** Pre-training gives a better starting point, but fine-tuning then has to travel from that point to a good classifier using the same weak gradient signal. Here fine-tuning is so under-powered that it cannot exploit the head start. A better initialization is worthless if the optimiser that follows it cannot move.

7 Conclusion

Greedy layer-wise pre-training is an initialization trick for deep sigmoid networks: train each layer as an autoencoder, stack the encoders, then fine-tune with labels. In this

experiment the pre-training phase measurably learned to reconstruct the hidden representations, but the mandated training budget (learning rate 0.001, 30 epochs, four width-3 sigmoid layers) is far too small for either the baseline or the pre-trained network to move off the majority-class prediction, so the two cases finish with identical 63.8% test accuracy. The takeaway is conceptual: pre-training helps a deep network find features it otherwise could not, but it cannot rescue a run whose fine-tuning step is itself starved of gradient. With modern choices (ReLU activations, a learning rate around 0.01 to 0.1, or a few hundred epochs) both networks would learn the task, and the gap between them would shrink further because good initialization matters most exactly when optimisation is hard.

Experiment 2: Regularization on Letter Recognition

8 Objective

- Train a $16 \rightarrow 32 \rightarrow 16 \rightarrow 26$ classifier on the UCI Letter Recognition dataset (20,000 examples, 16 numeric features per example, one of 26 capital letters as the label).
- Establish a baseline, then apply six well-known regularization or ensembling techniques, one per case.
- For every case record the final test loss, test accuracy, and the L_2 norm of the trained weights, then compare how well each technique generalises and how large its weights end up.

9 Theory

9.1 Generalisation, overfitting, underfitting

We care about performance on data the model has never seen. Two things can go wrong.

- **Overfitting:** the model memorises quirks of the training set. You see it as a widening gap where training accuracy keeps climbing while validation accuracy stalls or drops. Overfitting usually comes with large weights, because sharp decision boundaries need large numbers.
- **Underfitting:** the model has not learned enough yet, from too little capacity, too few epochs, or too small a learning rate. You see it as training and validation curves that are close together and both still improving when time runs out.

Regularization is a family of techniques that fight overfitting by making the model prefer simpler functions. The important caveat, which this experiment demonstrates, is that regularization only helps when you were overfitting in the first place. Applied to an underfitting model it just removes capacity the model still needed.

9.2 Softmax and sparse categorical cross-entropy

With 26 classes the output layer has 26 neurons. Softmax turns their raw scores z_k into a probability distribution,

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{26} e^{z_j}}, \quad \sum_k p_k = 1.$$

The loss is the negative log probability the model assigned to the correct class,

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log p_{i, y_i}.$$

“Sparse” just means the label is stored as an integer 0 to 25 rather than as a length-26 one-hot vector; the maths is identical. A model that guessed uniformly would score $\log 26 \approx 3.26$. The gradient of softmax combined with this loss is the famously clean $p - \text{onehot}(y)$.

9.3 The seven cases, each in a sentence or two

Case 1, baseline. The plain network. Everything else is measured against this.

Case 2, L_2 regularization. Add $\lambda \sum \|W\|^2$ to the loss (biases excluded), with $\lambda = 0.001$. The extra gradient term $2\lambda W$ pulls every weight slightly toward zero on each step, so the model keeps a weight large only if the data really pays for it. Result: smaller weights, smoother function.

Case 3, input noise. Add a small random perturbation to each training input every epoch (truncated Gaussian, standard deviation 0.1, clipped to $[-1, 1]$). The model is forced to give stable answers in a little neighbourhood around each example, which is a mild form of data augmentation and is mathematically close to L_2 .

Case 4, label noise. Randomly change 10% of the training labels to a wrong class. This is not a technique you would choose; it tests how gracefully the model degrades when the training labels are dirty.

Case 5, early stopping. Train for at most 30 epochs but watch the validation loss. If it fails to improve by at least 0.001 for 5 epochs in a row, stop and roll the weights back to the best epoch. The idea: the moment validation loss turns up is the moment the model starts overfitting, so freeze it there.

Case 6, ensemble. Train three networks with slightly different hidden sizes (32-16, 28-16, 32-12) independently, average their softmax probabilities, $P = (P_1 + P_2 + P_3)/3$, and predict the argmax of the average. Independent models make independent mistakes, so averaging cancels some of the error. This is the same reason the average of several noisy measurements is more reliable than any single one.

Case 7, dropout. During training only, randomly zero 50% of the units after H_1 and 20% after H_2 , rescaling the survivors so the expected sum is unchanged. No unit can rely on any other specific unit being present, so the network learns redundant, robust features. At test time dropout is switched off and the full network is used.

9.4 Why report the weight norm

The L_2 norm of all weight matrices stacked together, $\|W\|_2 = \sqrt{\sum_{\text{layers}} \sum_{ij} W_{ij}^2}$, is a simple proxy for how “aggressive” the learned function is. Techniques that regularise by shrinking the function (L2, dropout) should show a smaller norm than the baseline; techniques that do not (input noise, early stopping here) should look similar; an ensemble has a larger combined norm simply because it is three models.

10 Algorithm

1. Load `letter-recognition.data`. Map the letter A to Z to integers 0 to 25. Features are the 16 integer columns.
2. Make a 60:20:20 stratified train, validation, test split.
3. Fit `StandardScaler` on the training inputs only, transform all three splits.
4. For each case, build the network, train with the case-specific modification for 30 epochs (fewer if early stopping fires), tracking training and validation loss and accuracy each epoch.
5. After training, evaluate on the held-out test split and record test loss, test accuracy and $\|W\|_2$.
6. Plot training and validation loss and accuracy against epoch for each case, and collect all seven results into one table.

11 Implementation

The softmax-plus-cross-entropy gradient, written directly as p -onehot and averaged over the batch:

```
1 def scce_grad(probs, y):
2     grad = probs.copy()
3     grad[np.arange(len(y)), y] -= 1.0
4     return grad / len(y)
```

L_2 regularization is one extra term added to the weight gradient inside the optimiser step (note it never touches the bias):

```
1 def step(self, l2):
2     dW = self.dW + 2.0 * l2 * self.W if l2 else self.dW
3     self.vW = MOMENTUM * self.vW - LR * dW
4     self.vb = MOMENTUM * self.vb - LR * self.db
5     self.W += self.vW
6     self.b += self.vb
```

Dropout is applied between layers during the training forward pass and the same mask is reused in backward. The division by $(1 - \text{rate})$ is the rescaling that lets us leave the network untouched at test time:

```
1 def forward_dropout(layers, x, dropout, rng):
2     masks = {}
3     for i, layer in enumerate(layers):
4         x = layer.forward(x)
5         if dropout and i in dropout:
6             rate = dropout[i]
7             mask = (rng.random(x.shape) >= rate) / (1.0 - rate)
8             x = x * mask
9             masks[i] = mask
10    return x, masks
```

Input noise is resampled once per epoch and added to a shuffled copy of the training inputs, leaving validation and test untouched:

```

1 order = rng.permutation(n)
2 X_epoch, y_epoch = X_train[order], y_train[order]
3 if input_noise_sigma:
4     X_epoch = X_epoch + truncated_normal(input_noise_sigma,
5                                         X_epoch.shape,
6                                         clip=1.0, rng=rng)

```

Early stopping keeps the best weights seen so far and counts epochs without improvement:

```

1 if es["best_loss"] - val_loss > ES_MIN_DELTA:
2     es.update(best_loss=val_loss, best_epoch=epoch, wait=0,
3              best_snapshot=snapshot(layers))
4 else:
5     es["wait"] += 1
6     if es["wait"] >= ES_PATIENCE:
7         es["stop_epoch"] = epoch
8         break
9 # after the loop: restore(layers, es["best_snapshot"])

```

The ensemble simply averages probabilities from the three trained models:

```

1 p_test = np.mean([predict(m, X_test) for m in members], axis=0)
2 ensemble_acc = np.mean(np.argmax(p_test, axis=1) == y_test)

```

12 Output

Case	Test loss	Test accuracy	$\ W\ _2$
1. Baseline	2.3668	0.3788	15.84
2. L_2 , $\lambda = 0.001$	2.5123	0.3411	13.46
3. Input noise, $\sigma = 0.1$	2.3707	0.3808	15.80
4. Label noise, 10%	2.4345	0.3958	15.14
5. Early stopping	2.3668	0.3788	15.84
6. Ensemble (average of 3)	2.4110	0.4540	26.30
7. Dropout, 0.5/0.2	2.7064	0.3073	12.85

Table 3: All seven cases on the held-out test split. Uniform guessing would give accuracy $1/26 \approx 0.038$ and loss ≈ 3.26 .

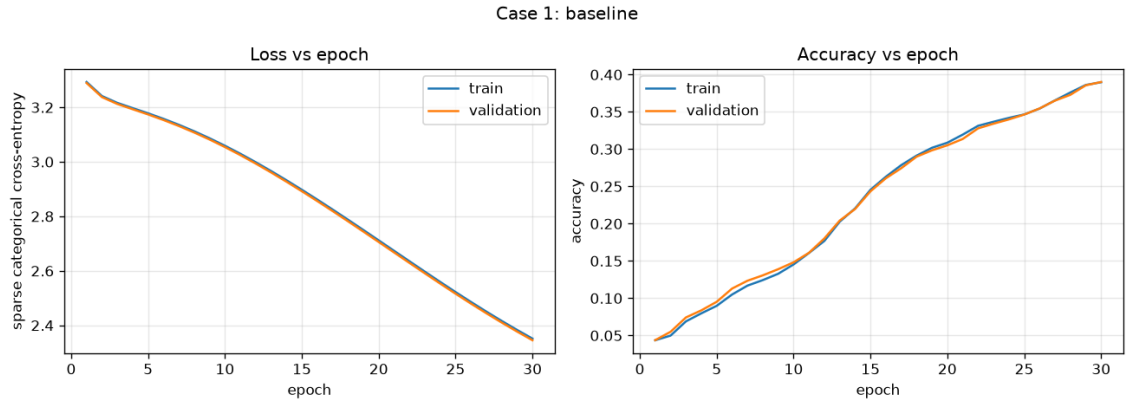


Figure 4: Baseline (the selected case). Training and validation curves lie almost on top of each other and are still climbing at epoch 30. This is the signature of underfitting.

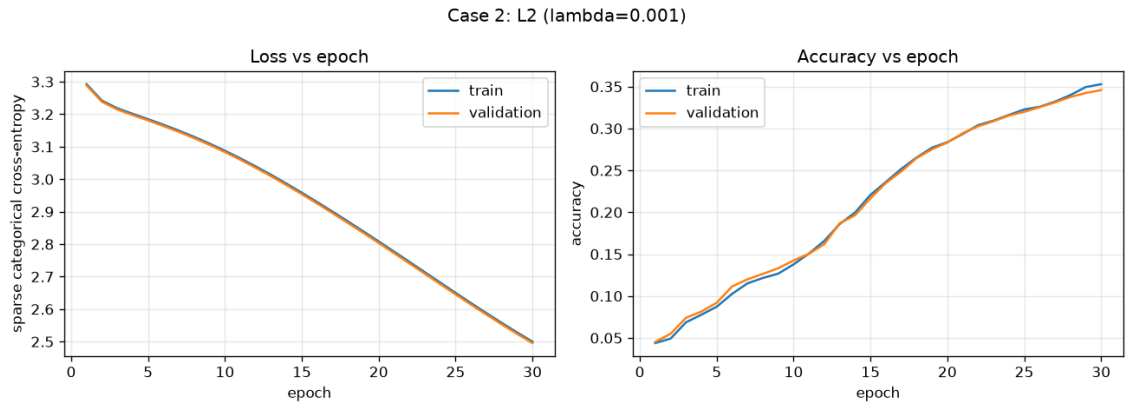


Figure 5: L_2 regularization. Same shape as the baseline but slightly worse, and the final weight norm is smaller (13.46 against 15.84).

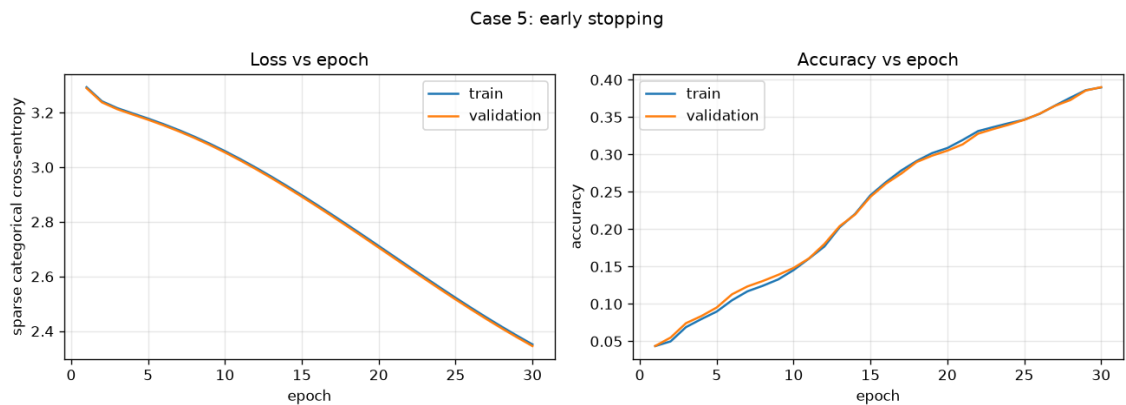


Figure 6: Early stopping. Validation loss decreases every single epoch, so the patience counter never reaches 5, the run goes the full 30 epochs, and zero epochs are saved.

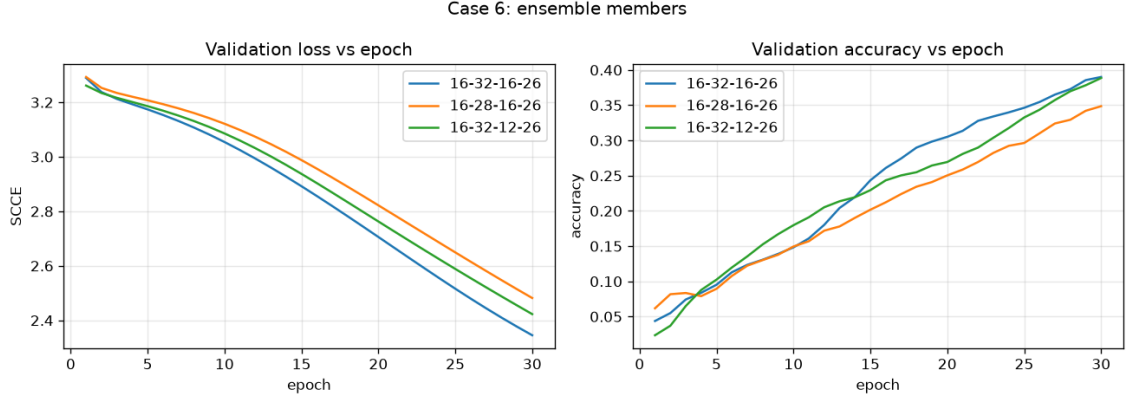


Figure 7: Ensemble members. Three individually mediocre networks whose averaged prediction beats all of them.

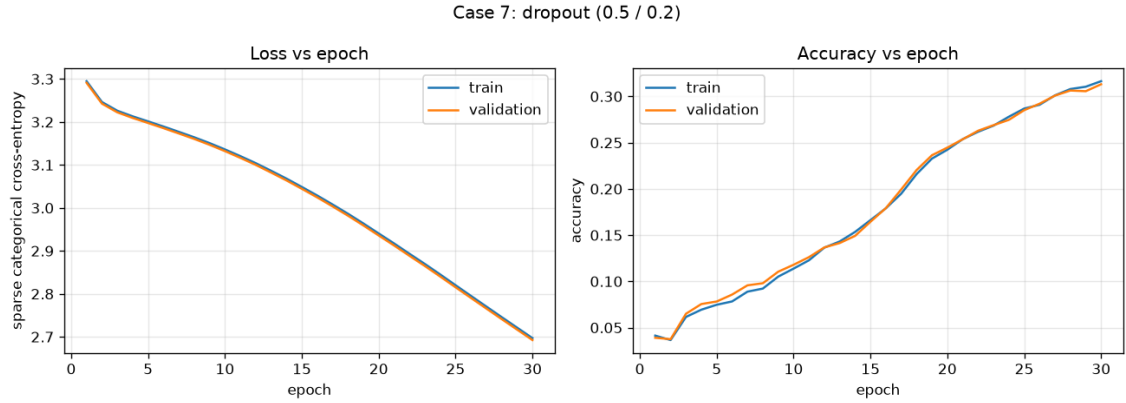


Figure 8: Dropout. The most heavily regularised case, the smallest weight norm (12.85), and the lowest accuracy. Dropout removed capacity the model still needed.

Ensemble member	Test accuracy	Test loss	$\ W\ _2$
16-32-16-26	0.3788	2.3668	15.84
16-28-16-26	0.3376	2.4988	14.76
16-32-12-26	0.3886	2.4322	14.94
average of the three	0.4540	2.4110	n/a

Table 4: Case 6 broken down. The average is 6.5 points above the best single member.

Early stopping details: best epoch = 30, stopping epoch = 30, epochs saved = 0.

13 Observation

- **Every model in this experiment is underfitting.** In all seven cases the training and validation curves are almost identical and both are still rising at epoch 30. There is no generalisation gap to close. With the fixed learning rate of 0.001, 30 epochs, and sigmoid activations, the network reaches only about 38% accuracy on a

task where a well-tuned MLP reaches roughly 90%. It has clearly learned something real (uniform guessing is under 4%), it just has not finished.

- **So the pure regularizers hurt, exactly as theory predicts.**
 - L_2 (Case 2) drops accuracy from 0.379 to 0.341. It spent gradient budget shrinking weights instead of fitting the data, and the weight norm duly fell to 13.46.
 - Dropout (Case 7) is worse still, 0.307, with the smallest norm of all, 12.85. Zeroing half the first hidden layer every step, in a network that is already too weak, just makes training slower.

Both techniques are doing their job correctly. Their job is simply not what this underfitting model needs.

- **The near-neutral cases behave as expected.** Input noise (Case 3) is tiny relative to standardized features, so it changes almost nothing: accuracy 0.381, norm 15.80, both within noise of the baseline. Early stopping (Case 5) is mathematically identical to the baseline here because validation loss never stops improving, so it never triggers; best epoch, stopping epoch and epochs saved confirm this.
- **Label noise (Case 4) did not hurt, and slightly helped.** Flipping 10% of training labels raised test accuracy a little, to 0.396. When a model is underfitting, the noisy labels act like a mild randomisation that keeps the optimiser moving, similar in spirit to adding jitter. On an overfitting model the same corruption would clearly lower accuracy.
- **The ensemble is the one clear win: 0.454, about 7.5 points above the baseline.** The three members make different errors because they have different architectures and different random initialisations. Averaging their probabilities cancels the errors that disagree and keeps the predictions they agree on. This works even though each member is individually weak, which is the key lesson: ensembling reduces the variance part of the error and does not require any single model to be good. Its combined weight norm is large (26.30) only because it is literally three networks.
- **Reading the weight-norm column.** It lines up neatly with how much each method restrains the function: dropout 12.85 < L_2 13.46 < label noise 15.14 < input noise 15.80 $\approx$ baseline 15.84 < ensemble 26.30. Smaller is not better here, because smaller came at the cost of fitting.

14 Conclusion

The table tells a consistent story once you notice that every model is underfitting. Regularizers that work by shrinking the model (L_2 , dropout) lowered both the weight norm and the accuracy, because they removed capacity a still-learning network needed. Regularizers that barely perturb the setup (small input noise, early stopping that never triggers) left the baseline essentially unchanged. Mild label noise slightly helped, for the same reason regularization hurt: the model had room to spare. The only technique

that improved generalisation was the ensemble, which raised test accuracy from 37.9% to 45.4% by averaging away the independent mistakes of three weak networks.

The broader lesson is about diagnosis before treatment. Regularization is the right tool when training accuracy is high and validation accuracy lags behind. When both are low and still rising, as here, the fix is more capacity, more epochs, a larger learning rate, or better activations, and adding regularization on top only slows the model down. Ensembling is the exception: because it attacks variance rather than capacity, it can help whether the base models are overfitting or underfitting.